Plano de consolidacao: Ortomapas, app de voo e simulador

- **Data:** 20/10/2026
- **Status:** plano e especificacao; o app mobile DJI e o simulador ainda nao estao implementados neste repositorio.
- **Banco de dados do produto:** PostgreSQL/PostGIS.
- **Escopo:** tres entregas integradas: Ortomapas Web, aplicativo mobile de campo/voo e simulador de aeronaves.

1. Resumo executivo

O Ortomapas nao e um unico app de controle de drone. A arquitetura alvo tem tres produtos que compartilham contratos, mas possuem responsabilidades diferentes:

1. **Ortomapas Web:** planeja projetos e missoes, guarda metadados, recebe fotos e telemetria, aciona NodeODM, apresenta produtos GIS e analises e oferece o copiloto.
2. **Ortomapas Mobile:** conecta-se localmente ao controle/aeronave DJI suportados, exibe video e telemetria, guia ou executa captura conforme capacidades comprovadas, roda inferencia visual no dispositivo e sincroniza dados com o Web.
3. **Bancada de simulacao e controle do drone futuro:** reproduz o modelo matematico, permite comparar PID/GA/RL, exibe a resposta no visualizador Matplotlib existente e prepara uma implementacao de controle candidata ao ESP32. E uma trilha de engenharia do drone proprio, nao apenas simulacao de rota do Ortomapas e nao uma copia do firmware do Mini 3.

O que existe hoje com evidencias e principalmente o **produto Web**: projetos/dados GIS, tarefas reais NodeODM, visualizacao 2D/3D, analises, planejamento/exportacao de missoes e partes do copiloto. Os planos do app mobile e do simulador sao especificacoes, nao implementacoes. Ainda nao ha app Android DJI, conexao fisica ao Mini 3, deteccao de gado em video real, analise mobile de lavoura nem simulador operacional integrado.

YOLO pode fazer **inferencia** de deteccao/segmentacao no telefone se o modelo for treinado e convertido para um runtime compativel. Isso nao significa que qualquer telefone sustentara o mesmo modelo, taxa de quadros, temperatura ou autonomia. A trilha correta e treinar/avaliar em GPU de estacao/servidor e implantar pesos otimizados no telefone. O ecossistema Android tem aceleradores e delegates diferentes por fabricante; cada perfil de aparelho precisa de benchmark. LiteRT: delegates NPU Android, exportacao mobile Ultralytics/LiteRT.

Tambem ha uma distincao importante para agricultura: o Mini 3 tem camera RGB, nao bandas NIR/red-edge. Ele pode alimentar indices RGB e modelos visuais, mas nao pode produzir NDVI/NDRE espectralmente valido sem sensor multiespectral e calibracao apropriados. Especificacoes oficiais DJI Mini 3.

2. Estado atual, sem confundir especificacao com entrega

Entrega	Implementado/validado	Falta para considerar operacional
Ortomapas Web: projetos, PostgreSQL/ PostGIS, produtos e mapas	Backend/frontend existentes e validacoes GIS documentadas	Fechar matriz integrada dos casos de uso, acesso por projeto, erros/estados e reprodutibilidade do seed; revisar lacunas identificadas nos documentos de analise
Visualizacao/analise GIS	Ortofoto e produtos de terreno reais; tiles/resultados persistidos para operacoes cobertas; visualizacao 3D e perfil foram corrigidos em fases anteriores	Cobrir operacoes e formatos pendentes, UX, limites de qualidade e aceite integrado de cada UC; nao generalizar evidencias de uma ferramenta para todas
WebODM/ NodeODM	Duas tarefas reais de 77 fotos concluídas via interface; ortomosaico, DSM, DTM, LAZ e relatorio importados e verificados	Importar/exibir malha OBJ texturizada; consolidar fluxo de projeto novo e aceite ponta a ponta/documentacao
Planejador de missao	Editor, AOI, exclusoes, grid/cobertura, validacao e CSV Litchi/KML/GeoJSON passaram por validacao web/PostGIS	Provar compatibilidade de arquivo no app alvo e missao com drone real. Exportar CSV nao prova voo/autonomia real

Entrega	Implementado/validado	Falta para considerar operacional
Copiloto web	Chat/contexto/memoria por usuario/projeto e ferramentas existem parcialmente; progresso registrado na Fase 8	Sumarizacao, segunda conta visualmente validada, aprovacoes de escrita e relatorio integrado; RAG esta deliberadamente depois
App mobile DJI	**Nao implementado**; ha estudo e especificacao	Spike com SDK, Mini 3, RC-N1, firmware e Android concretos; app, teste fisico e sincronizacao
YOLO mobile / gado / lavoura	**Nao implementado**. Existe pesquisa/agent de visao no servidor, mas isso nao e inferencia na camera do telefone	Dataset rotulado, modelo proprio, benchmark mobile, tracking/contagem, georreferencia e fluxo de revisao
Simulador/controle ESP32	Existem modelos 6-DOF, visualizacao Matplotlib, notebooks PID/GA e experimentos DDPG/TD3 no projeto local `drone`	Nucleo fisico validado, PID de referencia, GA executavel, RL que converge, exportacao/compatibilidade ESP32, HIL e calibracao real. Nos videos atuais o corpo aparece minusculo frente aos eixos de voo

Fontes de status internas: plano de melhorias e fases Web, NodeODM Fase 6, missoes Fase 7, Copiloto Fase 8, especificacao Android e simulador.

3. Arquitetura e fronteiras

```
Ortomapas Web / API / PostgreSQL + PostGIS / armazenamento de arquivos
       | contrato de projeto, missao versionada, upload, tarefas e produtos
       +------------------------------+
       |                              |
Ortomapas Mobile                Bancada de controle do drone proprio
DJI SDK + camera/video           fisica 6-DOF + PID/GA/RL
inferencias no dispositivo      Matplotlib 3D + exportacao ESP32/HIL
       |                              |
       +--------- mesmo contrato de missao/execucao --------+
```

- **Ortomapas e fonte de verdade** de projetos, usuarios/permissoes, AOIs, missoes aprovadas, voos, arquivos, analises e produtos publicados.
- **Mobile nao pilota via servidor.** DJI SDK e controle permanecem no telefone/controle local. A API serve sincronizacao, identidade, manifesto, alertas e retorno de produto.
- **Bancada do drone proprio e separada do adaptador DJI.** Ela executa uma dinamica 6-DOF, recebe setpoints/sensores simulados, roda controladores candidatos e visualiza estados/atuadores. A integracao com missoes Ortomapas e uma etapa posterior, por adapter e sem confundir esse controle com Virtual Stick DJI.
- **ODM e GIS ficam no servidor.** Fotos originais confirmadas podem iniciar NodeODM apos o voo. Inferencia no video pode ser ao vivo; ortomosaico/DSM/DTM finais nao sao produto instantaneo do Mini 3.
- O fluxo comum e: `projeto -> missao versionada -> voo real ou simulado -> eventos/fotos/telemetria -> sincronizacao -> processamento -> produto/analise -> revisao`.

4. Aplicacao 1: concluir o Ortomapas Web

4.1 O que deve ser fechado

1. **Requisitos e rastreabilidade:** congelar a lista canonica de casos de uso e mapear cada UC a rota, componente, tabela, dado de aceite, teste automatizado e evidencia visual. Rotular `implementado`, `parcial`, `nao iniciado` ou `bloqueado`; nenhum endpoint isolado vale como UC concluido.
2. **Projeto/dados/permissoes:** criar projeto por usuario, selecionar um projeto como contexto unico, cadastrar produtos por ID e origem, validar autorizacao nos downloads/tiles/analises e comprovar isolamento com duas contas.
3. **Catalogo GIS:** ortofoto, DSM, DTM, nuvem de pontos, malha 3D, vetores e derivados devem ter CRS, unidade, bounds, resolucao, origem, data, tipo e estado. Produto selecionado nunca deve ser substituido silenciosamente por um raster de entrada.
4. **Analises:** garantir entradas tipadas, CRS e unidade corretos, AOI quando aplicavel, saida propria persistida, estatisticas recalculadas, legenda coerente, tarefa/erro recuperavel e reabertura apos reload.
5. **ODM:** concluir importacao idempotente de todos os produtos suportados, acompanhar tarefa real, apresentar logs/qualidade/parametros e relacionar as saidas ao projeto e ao conjunto original de fotos.

6. **Planejador:** consolidar perfil de camera, GSD/footprint/overlap, altura de referencia, autonomia por bloco e validacao; manter exportacao bem identificada como arquivo de planejamento, nao como comprovacao de compatibilidade com DJI.
7. **Copiloto:** concluir Fase 8 (sumarizacao, segunda conta, escrita com aprovacao e relatorio). Preservar memoria por usuario/projeto, origem de fatos, falhas como falhas e RAG para fase posterior.
8. **Aceite operacional:** rodar suite completa contra Postgres/PostGIS, NodeODM e LM Studio reais quando disponiveis; criar projeto limpo, usar dados com manifesto, verificar API/DB/arquivos/telas e produzir relatorio com limites reais.

4.2 Ordem de fechamento Web

Etapa	Entrega	Evidencia de saida
W0	Atualizar matriz UC/requisito/API/UI/dado/teste e backlog de gaps	Matriz assinada por teste: nada fica “pronto” sem resultado verificavel
W1	Fechar P0 de autorizacao, produto/tiles, unidades e persistencia	Testes de referencia espacial + duas contas + reabertura
W2	Fechar catalogo, menus, upload, erros e estados	Playwright com projeto vazio, projeto com produtos e tarefas com falha
W3	Completar produtos WebODM, visualizadores e analises pendentes	Dados reais, tarefa NodeODM, 2D/3D, exportacao e hashes
W4	Fechar missao e contrato mobile/simulador	OpenAPI versionada e testes de round-trip JSON/KML/GeoJSON
W5	Fechar Copiloto Fase 8; depois aceite integrado	Duas contas, contexto retomado, confirmacao de escrita, timeout e relatorio

5. Aplicacao 2: Ortomapas Mobile, voo e analise visual

5.1 Escopo do MVP

Android nativo em Kotlin com DJI Mobile SDK (MSDK) V5 como trilha inicial, primeiro para **observacao e captura guiada**, nao para assumir autonomia DJI antes da prova fisica. A documentacao DJI apresenta MSDK V5 como SDK Android e inclui Mini 3 em tabelas de suporte/firmware, mas a lista e combinacao de suporte variam por versao; a matriz exata de SDK, firmware, controle, telefone e API deve ser travada no Spike M0. DJI MSDK, lista e combinacoes de produtos.

5.2 Funcoes de voo e captura

- Parear/inicializar SDK, descobrir aeronave/controle/firmware e consultar capacidades; nao habilitar uma funcao ausente.
- Baixar uma missao publicada e imutavel do projeto: missao/versao/hash, AOI, exclusoes, altitude e referencia, rota, camera, GSD, sobreposicoes, velocidade, gimbal, intervalo/foto, bateria-reserva e limites operacionais.
- Pre-voo: identidade do piloto, firmware/capacidade, GNSS, bateria, enlace, camera/cartao, gimbal, espaco livre, geofence/AOI, previsao, RTH e area de seguranca; bloqueio de condicao critica e decisao final do piloto.
- Na primeira entrega: mapa, video, telemetria e instrucoes de proximo trecho/foto; registro local offline de cada comando/evento e controle manual prioritario.
- So depois do Gate de bancada/campo: adicionar waypoint embarcado do SDK se for comprovado no Mini 3/firmware concreto. Virtual Stick fica em trilha separada, com watchdog, fail-safe definido, teste de perda de link e aceite de seguranca; nao usar como substituto implicito.
- A captura gera `photo_event_id`, timestamp UTC e monotonic, waypoint/trecho, pose/telemetria, camera settings, resultado do comando, identificador/EXIF do arquivo e hash. Separar foto planejada, comando enviado, arquivo encontrado, arquivo validado e arquivo sincronizado.

5.3 Analise em tempo real: gado

Caso de uso: durante uma passagem, marcar animais visiveis, estimar contagem por quadro/trecho, evitar contar repetidamente o mesmo animal em quadros consecutivos, sinalizar baixa confianca e permitir revisao humana no mapa/miniatura.

Pipeline proposto: decodificador do video DJI -> fila limitada de frames (nao bloquear voo) -> pre-processamento -> YOLO detection/segmentation -> tracking temporal -> associacao a telemetria/camera -> agregacao por trecho/AOI -> overlay/alerta local -> armazenamento leve e sincronizacao posterior.

- Detecao por frame nao e contagem de individuos unicos: contagem operacional requer tracking entre frames, zonas/linhas de passagem e regras contra duplicacao. Sem tracking calibrado, apresentar “deteccoes observadas”, nao “numero de animais no pasto”.
- Cada resultado inclui especie/classe, bbox ou mascara, confianca, timestamp, frame, trecho/AOI, versao/hash do modelo e estado (`provisorio`, `revisado`, `rejeitado`).
- O modelo sera treinado/ajustado em dataset rotulado de imagens reais de drone do alvo (gado, cavalo, pessoas/veiculos se relevantes), com variacao de altura, angulo, tamanho, horario, sombra, vegetacao e camera. Separar treino/validacao/teste por voo, propriedade e data para evitar vazamento de quadros quase identicos.
- Metricas de aceite: precision/recall e mAP por classe; erro absoluto medio de contagem por trecho; duplicacoes/omissoes; estratificacao por altura/condicao; latencia end-to-end p50/p95; FPS efetivo, temperatura, throttling, memoria, bateria e frames descartados.
- Inicialmente usar uma amostra de video configuravel e fila curta; descartar frames antigos quando houver atraso. Inferencia nunca altera controle/rota nem substitui o piloto. Se o aparelho aquecer ou perder frames, degradar para menos amostras/pausar analise e manter captura/voo.
- O treinamento recorrente deve ocorrer na estacao/servidor com GPU apropriada; o telefone e primariamente executor de inferencia. Exportar modelo menor/quantizado e comparar numericamente contra o modelo de referencia antes de publicar.

5.4 Analise de lavoura e espectral

Separar tres niveis para nao chamar tudo de “espectral”:

Nivel	Sensor/dados	Analises possiveis	Limite
RGB imediato (Mini 3)	Video/foto visivel	Contagem, cobertura visual, cor/textura, indices RGB como VARI/TGI/ExG/GLI, deteccao visual de falhas/anomalias se modelo treinado	Nao mede NIR/red-edge; luz, sombra, exposicao e balanco de branco afetam indices
Multiespectral	Camera com bandas calibradas (ex. blue/green/red/red-edge/NIR), sincronizacao entre bandas e referencia radiometrica	NDVI/NDRE e indices definidos pelo sensor, vigor relativo, zonas de variabilidade	Exige sensor compativel, calibracao/refletancia e pipeline radiometrico; nao e capacidade da camera RGB do Mini 3
Hibrido mobile + GIS	Inferencia resumida no telefone e originais/productos no Ortomapas	Alertas em campo; mapas/zonas quantitativas e comparacao temporal depois do voo	Resultado ao vivo e preliminar; ortorretificacao e produtos finais continuam no servidor

Fluxo agronomico: escolher cultura/estagio e protocolo -> capturar com exposicao e sensor documentados -> QA de dados -> inferencia/indices -> georreferenciar e revisar -> sincronizar -> ortorretificar/normalizar no servidor quando aplicavel -> comparar campanhas e exportar zonas. Rótulos como estresse hidrico/doenca precisam de validacao agronomica e ground truth; modelo RGB nao diagnostica causa com certeza.

5.5 Plataforma mobile e limites

- Android + MSDK DJI e prioridade porque o SDK V5 documentado e Android. iOS pode receber arquivos e executar modelos Core ML, mas **nao entra como plataforma de controle do Mini 3** sem comprovar um SDK DJI iOS que suporte este drone e este fluxo. MSDK V5 atual nao deve ser presumido multiplataforma.
- Inferencia no dispositivo pode usar LiteRT/GPU/NPU no Android; delegate e desempenho variam com SoC, drivers e operadores. Manter fallback CPU, matriz minima de aparelhos, teste de aquecimento e resultado visivel da capacidade efetivamente ativa. LiteRT NPU.
- Para iOS/Core ML existe trilha de exportacao, mas isso comprova formato de inferencia e nao controle DJI. Ultralytics Core ML.
- Avaliar a licenca do framework/pesos e direitos dos dados antes de distribuir produto proprietario/comercial; o plano nao escolhe uma licenca em nome do usuario.

5.6 Etapas do app mobile

Etapa	Trabalho	Gate/validacao
M0	Inventariar Mini 3, RC-N1, firmware, cabo, Android, DJI App Key, SDK/ licencas e capacidades de camera/video/missao/midia	Amostra DJI reconhece o equipamento; tabela de capacidades e erros arquivada
M1	App de observacao: conexao, telemetria, video, estado e diario local	Sessao longa de bancada, reconexao, sem crash e sem voo autonomo
M2	Pacote de missao versionado, mapa, pre-voo e captura guiada	Mesmo hash/rota no Web e telefone; offline/restart; fotos reconciliadas
M3	Prova de camera/midia e sincronizacao resumivel	Captura real, EXIF/hash, falha/retry sem duplicar, fotos associadas a projeto
M4	YOLO gado em dataset piloto	Ground truth, metricas por condicao, demonstracao ao vivo e revisao humana
M5	RGB agricultura e configuracao multiespectral futura	RGB declarado como RGB; NDVI bloqueado sem NIR; dados radiometricos rastreaveis
M6	Modo automatico aprovado para o modelo exato ou guia operacional robusto	Gate fisico autorizado, perda de enlace, pausa/RTH e controle manual aprovados
M7	Sincronizacao final, disparo NodeODM e retorno ao Web	Execucao, manifestos, imagens, QA, produtos e analises ligados aos mesmos IDs
M8	Matriz de aparelhos, robustez, distribuicao e privacidade	Soak/thermal/battery, permissao, criptografia local, logs redigidos e release assinado

6. Aplicacao 3: bancada de simulacao e controle parametrizavel

6.1 Objetivo e o que nao promete

O objetivo primario e desenvolver e comparar o controle do **quadricoptero proprio, cujo controlador alvo e ESP32**. A bancada deve simular a planta fisica, permitir escolher `sem controle`, `PID baseline`, `PID sintonizado por algoritmo genetico` ou `RL experimental`, animar o estado no Matplotlib ja existente e registrar os sinais para comparar. O controlador Python de referencia e os parametros/algoritmos aprovados devem depois ser portados ou gerados como C/C++ para o ESP32 e comparados em vetores comuns. A integracao de rotas do Ortomapas e secundaria a essa bancada.

O levantamento em `/mnt/data1/progpython/drone` encontrou ativos seus: modelo 6-DOF/RK4 e desenho 3D em `01_RLDDPGTD3LL/TD3DRONE.ipynb`; a fase inicial e controladores em `03_rldrone`; conversas historicas sobre dinamica/ PID/TD3; e o projeto de terceiros `02_drone_sim` (licenca MIT), que e referencia, nao codigo seu. A documentacao local `DOCUMENTACAO_DRONE.md` registra que o TD3 no drone nao convergiu e identifica bugs concretos. Reaproveitar os notebooks como fonte de requisitos/casos de regressao, nao importar como biblioteca validada.

Achado especifico do PID genetico: `03_rldrone/rldroneeulerPID.ipynb` contem um bloco DEAP com populacao 50, 50 geracoes, cruzamento/mutacao e busca de ganhos, mas a celula tem `execution_count: null` e nenhuma saida. A versao de 3 genes aplica os mesmos Kp/Ki/Kd a altitude, roll e pitch; `simulate_drone()` devolve somente historico de altitude e o fitness compara altitude a zero, portanto nao avalia estabilidade angular, rastreamento nem saida dos quatro motores. A variante comentada de nove ganhos tem trecho de codigo quebrado. A documentacao tambem marca indefinicao de `dt`, limites de thrust e variaveis de tuning, e o loop nao chama a dinamica. Portanto, existe **tentativa/documentacao** do GA, nao sintonia genetica comprovadamente funcional.

Inspecao visual de um artefato gravado: extraí um quadro aos 4 s de `01_RLDDPGTD3LL/video/TESTEEVDRONE.mp4` e de `video/Drone-v1_episode_0.mp4`. O grafico 3D e a grade estao ativos, mas o modelo do drone aparece como uma cruz/ponto vermelho quase imperceptivel no volume de eixos aproximado de X=40 m, Y=20 m, Z=120 m. A cena prova que existe renderizacao, nao que a atitude, dimensao ou queda possam ser inspecionadas com clareza. O viewer precisa separar escala de desenho (exagero visual configuravel) de coordenadas/medidas fisicas; manter uma trilha global e uma vista local/ inset do corpo com eixos, orientacao, vetores e motores.

O arquivo `03_rldrone/rldroneeuler.py` tambem nao serve como planta validada: ele injeta aceleracao e giroscopio aleatorios nos estados, calcula empuxo apenas em Z, nao calcula aceleracao angular a partir do torque e mistura estados verdadeiros com leituras de sensor. Ja `03_rldrone/rldrone3.py` e um modelo simplificado de seis estados com mapeamento de acoes, unidade de torque/massa e visualizacao com problemas. A base mais promissora e a versao mais

nova do notebook 6-DOF em `01_RLDDPGTD3LL`, mas sua fisica/recompensa/TD3 tambem precisam passar pelos testes listados na documentacao local antes de treinamento.

6.2 Modelo de dados e simulacao

- `DroneProfile`: massa, tensor de inercia, geometria, motor/atuador, limites de empuxo e taxa, bateria/consumo, controlador configurado, carga e limites de seguranca. Cada parametro tem unidade SI, fonte, incerteza, faixa e versao.
- `SensorProfile`: GNSS, IMU, barometro, magnetometro, camera/gimbal, latencia, ruido, bias, atualizacao e falhas.
- `EnvironmentProfile`: terreno/DEM, vento medio/rajada, densidade/ar, geofence/obstaculos e coordenadas locais ENU/NED.
- `Mission`: contrato versionado comum com Ortomapas; waypoints, altitude/referencia, velocidade, rumo, gimbal, captura, AOI/exclusoes e blocos.
- `SimulationRun`: seed deterministica, perfil/modelo e hash da missao, parametros efetivos, telemetria, eventos, fotografias sintéticas/footprints e resultados.
- Dinamica rigid-body 6-DOF com atitude em quaternion, visualizacao Euler, integracao de passo fixo deterministica (RK4 ou comparavel), motor/ESC/controlador desacoplados. A planta recebe comandos do controlador e produz estado fisico; sensores simulados leem esse estado com ruido/bias/latencia parametrizados, sem substituir estado verdadeiro por ruido aleatorio.
- API comum de controle em Python: `reset(config)`, `update(sensor_sample, setpoint, dt) -> actuator_command`. O mixer transforma comandos coletivos/roll/pitch/yaw em quatro comandos motores conforme geometria e ordem real dos motores configurada. Saturacao, limites, anti-windup e failsafe fazem parte do contrato.
- Variaveis visiveis e gravadas: setpoint, estado verdadeiro, IMU simulada, erro, P/I/D por eixo, saida do mixer, RPM/PWM/DShot comandado, limites/saturacao, bateria, forcas/torques, recompensa do RL e tempo de execucao do loop.
- Injetar vento, erro de sensor, atraso/jitter, bateria baixa, camera sem captura, GNSS degradado e perda de enlace; testar pausas, abortar, RTH/pouso simulado e relatorio de desvio/cobertura.
- UI com mapa e cena 3D simultaneos: aeronave, atitude, rota planejada/real, pegadas fotografadas (simuladas), envelope de camera, telemetria, graficos e eventos. Render de foto sintetica deve ser identificado como sintetico; nao misturar com produtos ODM reais.

Se o drone futuro tiver autopiloto PX4, avaliar posteriormente um backend SITL/Gazebo para exercitar o autopiloto real virtualizado; se usar controlador proprio, manter motor 6-DOF e adapter especifico. Esta decisao depende da arquitetura concreta do drone, nao deve atrasar o simulador de missao. O simulador oficial que existe no DJI MSDK e util para validar chamadas/estados suportados pelo SDK, mas e ferramenta distinta do nosso simulador fisico configuravel. DJI MSDK e SimulatorManager.

6.3 Etapas da bancada e do simulador

Etapa	Trabalho	Gate/validacao
S0	Auditar os notebooks/scripts, confirmar estado do Matplotlib, dependencias, licenca, parametros, convencoes e codigo de firmware ja testado no ESP32	Inventario de artefatos com caminhos/commit/datas e lista de inconsistencias; nenhum codigo externo copiado
S1	Corrigir/extraír uma planta 6-DOF deterministica; estado, referenciais, gravidade, forcas, torques, motor/ESC e sensores separados	Testes de hover, queda livre, sinal dos motores/eixos, energia/unidades, passo temporal e repetibilidade antes do controlador
S2	Recuperar e melhorar o visualizador Matplotlib 3D como instrumento de engenharia; corrigir corpo quase invisivel na escala global	Visual mostra corpo/eixos legiveis com exagero grafico declarado (fisica inalterada), atitude, trilha, setpoint, vetores de empuxo/gravidade, RPM dos quatro motores, vento, estados e curvas temporais; reproduz caso “sobe e cai sem controle” por configuracao conhecida
S3	Fazer PID baseline de cascata (taxa angular -> atitude; altitude/posicao como loops externos), com mixer, limites, anti-windup e falhas	Pairar, resposta a degrau, perturbar cada eixo, saturar e recuperar; medir overshoot, settling time, erro estacionario e estabilidade
S4	Tornar o GA reprodutivel para sintonia offline dos ganhos PID, com limites fisicos e fitness multiobjetivo	Fitness calcula atitude/altitude/taxa, energia/atuacao, saturacao e sucesso; multiplas seeds e cenarios; conjunto final avaliado em cenarios nunca usados na busca

Etapa	Trabalho	Gate/validacao
S5	Corrigir ambiente e pipeline TD3/SAC para experimento de RL, primeiro contra os testes/ modelos classicos	API Gymnasium consistente, observacoes normalizadas, acoes normalizadas, recompensa de tracking/seguranca, seeds/randomizacao, checkpoint; curvas de aprendizagem e avaliacao independente contra PID
S6	Comparar PID baseline, PID+GA e RL na mesma bateria de cenarios e no visualizador	Tabelas e curvas equivalentes; RL fica `experimental` se nao superar criterios predefinidos e nao substitui o controlador de seguranca
S7	Exportar configuracao PID validada e um controlador C/C++ candidato ao ESP32; considerar exportacao da politica RL somente se couber e atender o deadline	Testes unitarios de firmware por vetores de entrada/saida iguais ao Python, limites de memoria/stack, tempo WCET e periodo de controle medidos no modelo ESP32 concreto; nunca flash/armar automaticamente
S8	Software-in-the-loop e depois hardware-in-the-loop sem helices, com IMU/firmware/telemetria por serial e corte fisico	Conferir taxa real, atrasos, saturacao, perda de pacote, watchdog, fail-safe e consistencia Python/C++; seguranca de bancada documentada
S9	Calibrar parametros com bancada de empuxo, caracterizacao motor/ESC/bateria e logs de voo autorizado; opcionalmente ligar missao/area do Ortomapas ao simulador	Relatorio erro modelo-medicao por condicao; somente depois considerar voo progressivo e integracao de rota. Simulacao nunca certifica voo

O Matplotlib existente e primeira interface obrigatoria, nao um mock. Em uma fase posterior, a bancada web/3D pode consumir os mesmos registros, mas nao deve atrasar a validacao do modelo, do controle nem a comparacao Python-versus-ESP32.

6.4 Como o controle chega ao ESP32

O alvo nao e “exportar o agente e torcer para funcionar”. O processo seguro e: (a) manter controle de referencia Python simples e deterministico; (b) testar/otimizar nele; (c) gerar um arquivo de configuracao de ganhos e, se necessario, uma implementacao C/C++ versionada; (d) executar os mesmos vetores em Python e firmware e comparar tolerancias; (e) medir tempo maximo de execucao no ESP32 e jitter no periodo de controle; (f) testar HIL sem helices, watchdog e corte; (g) so entao planejar teste fisico progressivo. A geracao nunca grava/flasha firmware nem arma motores sem acao humana.

PID e GA sao trilha prioritaria porque o firmware ESP32 precisa de um loop previsivel: o GA roda offline e entrega ganhos, nao executa no drone. RL e uma linha de pesquisa separada: treinar na simulacao, avaliar robustez e gap sim-to-real, exportar uma politica pequena somente apos validar memoria, latencia, saturacao e supervisor seguro. Inicialmente, o PID aprovado continua como controlador/fallback; uma politica RL nao aprovada nao controla hardware.

7. Contrato que volta para o Ortomapas

Cada execucao real ou simulada deve ter identidade inequívoca e dados associados a projeto/usuario. Esquema conceitual (nomes finais dependem de migracao e OpenAPI):

- `flight_execution`: `execution_id`, `project_id`, `mission_id`, `mission_version`, `mission_hash`, `mode` (`real`/`simulated`), `aircraft_profile`, `app_version`, `sdk/firmware`, `operator_id`, estado, inicio/fim, origem e observacoes.
- `flight_event`: sequencia idempotente, timestamp monotonic/UTC, tipo, coordenada, altitude/referencia, origem e payload validado (armar, decolar, foto, alerta, pausa, perda de link, RTH, pouso).
- `telemetry_sample`: posicao/atitude/velocidade/bateria/GNSS/sinal/gimbal; politica de retencao e particionamento antes de alta frequencia.
- `media_manifest`: foto/video, hash, tamanho, MIME, EXIF, camera, `photo_event_id`, estado de transfer, chunks recebidos e resultado de verificacao.
- `vision_observation`: classe, bbox/mascara, confianca, timestamp/frame, geometria estimada e incerteza, `model_id/version/hash`, metricas, estado de revisao e link para original/thumbnail.
- `processing_job` e `product`: UUID externo NodeODM, parametros efetivamente usados, estado/log/erros, produto/ tipo/CRS/resolucao/bounds/hash e ligacoes de origem.
- `model_registry`: tarefa/classe, dataset/version/hash, treino/avaliacao, export/runtime, licenca, dispositivos testados e status `experimental`/`approved`.

Upload offline deve ser resumivel, checksummed e idempotente. Armazenar binarios em armazenamento de arquivos/objeto e metadados/controle/acesso no PostgreSQL/PostGIS. Telemetria de simulacao e deteccoes preliminares nunca devem se apresentar como medicao ou produto fotogrametrico validado.

8. Roadmap integrado, dependencias e prioridade

1. **Consolidar Web (W0-W2)**: matriz real de UCs, gaps P0, autenticacao/permissoes, produto/mapa/3D confiaveis e seed limpo. Evita levar bugs de contexto para campo.
2. **Contrato partilhado (W4 + M0)**: definir OpenAPI/JSON Schema, versionamento, pacote de missao/hash, execucao/eventos/manifesto/fotos/modelos; testar clientes simulados antes do SDK.
3. **Spike fisico M0**: Mini 3/RC-N1/firmwares/Android/App Key e limites SDK; decidir captura guiada, waypoint, Virtual Stick e midia apos dados, nao suposicao.
4. **Mobile M1-M3**: conexao, video, telemetria, missao aprovada, captura guiada, journal e sincronizacao; sem YOLO antes de medir concorrencia de video/SDK e termica.
5. **Bancada S0-S3 em paralelo**: validar dinamica, reproduzir visualmente o caso sem controle e estabelecer PID baseline antes de otimizar ou integrar missao.
6. **Visao M4-M5**: dataset/ground truth -> baseline YOLO -> teste offline no telefone -> overlay ao vivo -> tracking/contagem -> RGB e trilha multiespectral independente.
7. **Voo automatico M6**: gate de seguranca separado e ensaio autorizado com piloto; se Mini 3 nao suportar caminho seguro/necessario, manter captura guiada ou definir outro equipamento.
8. **Integracao final W3/W5/M7/S6**: ciclo projeto novo -> missao -> real/simulado -> upload -> NodeODM -> produtos 2D/3D -> analise/contagem -> revisao/exportacao e relatorio por UC.
9. **Operacao M8/S7**: dispositivos suportados, assinaturas, logs/metricas, retencao/privacidade, treinamento de usuario, suporte e calibracao com voos reais.

Caminho critico

`M0 hardware/SDK -> contrato de missao e capacidades -> M1 observacao -> M2/M3 captura/midia -> M4 dataset/inferencia -> M7 integracao ODM`. O simulador usa contrato de missao em paralelo e antecipa testes de geometria/runner, mas nao remove gates DJI/campo. O fechamento Web W0-W2 e requisito para iniciar projetos e upload mobile sem risco de cruzar dados.

9. Criterios de aceite do produto completo

- **Caso de uso:** cada UC tem requisito, pre-condicao, usuario/permissao, dados, passos, saida esperada, assert de API/DB/arquivo/UI e evidencia; contagem de telas nao e criterio de aprovacao.
- **Voo real:** conjunto e firmware identificados, controle manual prioritario, checklist aprovado, comportamento de link/bateria demonstrado em ensaio autorizado, diario completo e fotos reconciliadas. Simulador nao substitui isso.
- **Contagem de gado:** ground truth separado do treino, metricas por altura/cenario, contagem por individuo/trecho com tracking e revisao, taxa de falso positivo/negativo comunicada e limitacao de visibilidade declarada.
- **Agricultura RGB:** indices nomeados corretamente, metadados de captura e validacao visual/agronomica; nenhum NDVI/NDRE sem banda exigida e calibracao registrada.
- **Inferencia no aparelho:** matriz explicita de aparelho/OS/runtime, latencia p95, throughput, descarte de quadros, bateria/temperatura em sessao longa e fallback; nenhum requisito “qualquer celular” sem ensaio.
- **Simulador/controle:** determinismo, unidades, frames, sinais e planta comparados a referencias; PID/GA/RL avaliados com a mesma bateria; viewer Matplotlib mostra sinais/estado/atuadores; C++ ESP32 passa parity, deadline, limites e HIL sem helices. Simulacao sozinha nao certifica dinamica real.
- **Integracao:** IDs iguais ligam projeto, missao/versao, execucao, evento, foto, observacao, task NodeODM e produto; upload repetido nao duplica e toda tela identifica dado real, sintetico ou estimado.

10. Decisoes recomendadas

1. Manter o **PostgreSQL/PostGIS existente** como banco do Ortomapas. WebODM pode manter schema/banco/papel separado na mesma instancia se necessario, ligado por API e identificadores externos; nao duplicar servidor sem motivo operacional.

2. Priorizar **Android nativo/Kotlin** para controle DJI; deixar UI cross-platform para depois do spike e nao prometer iOS para pilotar Mini 3.
3. Usar **YOLO para inferencia visual** de gado/objetos; treinar e validar modelos com GPU em maquina de desenvolvimento/servidor, depois converter/quantizar para cada runtime/dispositivo. No celular: inferencia, nao treino como fluxo de producao.
4. Tratar gado como deteccao + tracking + contagem por zona/trecho, com revisao, nao simples soma de boxes de cada frame.
5. Separar analise RGB imediata de analise multiespectral agronomica. Para NDVI/NDRE, planejar camera/bandas e calibracao, possivelmente outro drone/sensor.
6. Fazer simulador generico/configuravel para o drone futuro; perfil Mini 3, se feito, sera aproximado e rotulado. Usar projeto `drone` como referencia/testes apos revisao, nao como biblioteca validada.
7. Fazer a proxima implementacao no Web ser **W0: matriz de casos de uso e gaps integrada**. Paralelamente, iniciar somente o **M0 de viabilidade de bancada** e **S0 de auditoria**, sem comandos de voo ou claims de fidelidade.

11. Referencias tecnicas

- DJI MSDK V5 - documentacao e plataformas/produtos
- DJI MSDK - lista de produtos e versoes de firmware
- DJI Mini 3 - especificacoes oficiais da camera
- Google LiteRT - delegates NPU em Android
- Ultralytics - exportacao LiteRT/TFLite
- Ultralytics - Core ML